ONLINE JOB PORTAL SYSTEM

A Project Submitted

To

SHRI SHANKARACHARYA PROFESSIONAL UNIVERSITY,
BHILAI (C.G), INDIA

In Partial Fulfilment

For The Award Of The Degree

Of

BACHELOR OF COMPUTER APPLICATION

In

Computer Science and Technology

By

AYUSHI MANJHI
BHANUJA AMBADE
MAHEK BANO

Enrolment no: SSPU202300032, SSPU202300034, SSPU202300073

University Roll no: 2223001032, 2223001034, 2223001073

Under The Guidance of

Mr. Deepak Singh

Assistant Professor

Computer Science and Technology

Session: 2024-2025

ONLINE JOB PORTAL SYSTEM

A Project Submitted
To



SHRI SHANKARACHARYA PROFESSIONAL UNIVERSITY, BHILAI (C.G), INDIA

In Partial Fulfilment

For The Award Of The Degree

Of

BACHELOR OF COMPUTER APPLICATION

In

Computer Science and Technology

By

AYUSHI MANJHI
BHANUJA AMBADE
MAHEK BANO

Enrolment no: SSPU202300032, SSPU202300034, SSPU202300073

University Roll no: 2223001032, 2223001034, 2223001073

Under The Guidance of

Mr. Deepak Singh

Assistant Professor

Computer Science and Technology

Session: 2024-2025

Declaration by the Candidate

I the undersigned solemnly declare that the report of the Project work entitled **"ONLINE JOB PORTAL SYSTEM",** is based on my own work carried out during the course of my study under the supervision of **Deepak Singh.**

I assert that the statements made and conclusions drawn are an outcome of the project work. I further declare that to the best of my knowledge and belief that the report does not contain any part of any work which has been submitted for the award of any other degree/diploma/certificate in this University/deemed University of India or any other country. All helps received and citations used for the preparation of the Project have been duly acknowledged.

Name of the Candidate : Ayushi Manjhi
Bhanuja Ambade
Mahek Bano
Roll No. : **2223001032**
2223001034
2223001073

Enrollment No.: **SSPU202300032**
SSPU202300034
SSPU202300073

Signature of the Supervisor
Name of the Supervisor : Mr. Deepak Singh
Designation of the Supervisor : Assistant Professor
Name of the Department : Computer Application
Name of the Institute with address :
Shri Shankaracharya Professional
University, Junwani Smriti Nagar,
Bhilai(C.G.)

Certificate of the Supervisor

This is to certify that the report of the Project entitled "**ONLINE JOB PORTAL SYSTEM** " , is a record of bonafide research work carried out by **Ayushi Manjhi, Bhanuja Ambade, Mahek Bano** bearing Roll No. : **2223001032, 2223001034, 2223001073,** & Enrollment No. : **SSPU202300032, SSPU202300034, SSPU202300073** under my guidance and supervision for the award of Degree of **Bachelor of Computer Application** in the faculty of **Computer Science and Technology**, of Shri Shankaracharya Professional University, Bhilai (C.G.), India. To the best of my knowledge and belief the Project

- Embodies the work of the candidate him/herself,
- Has duly been completed,
- Fulfills the requirement of the ordinance relating to the B.Tech degree of the University and is up to the desired standard both in respect of contents and language for being referred to the examiners.

Signature of the Candidates
Name : Ayushi Manjhi
Bhanuja Ambade
Mahek Bano
Department : Computer Science and Technology

Forwarded to Shri Shankaracharya Professional University, Bhilai

Signature of the Director/Principal

Name & Address of the Institute :
Shri Shankaracharya Professional University, Junwani Smriti Nagar, Bhilai(C.G.)

Certificate by the Examiners

The Project entitled **"Title of the Project**" Submitted by **Ayushi Manjhi, Bhanuja Ambade, Mahek Bano** bearing Roll No. : **2223001032, 2223001034, 2223001073,** Enrollment No. : **SSPU202300032, SSPU202300034, SSPU202300073** has been examined by the under signed as a part of the examination and is hereby recommended for the award of the degree of **Bachelor of Computer Application** in the faculty of **Computer Science and Technology** of Shri Shankaracharya Professional University, Bhilai.

Internal Examiner
Date:

External Examiner
Date:

Abstract

The advancement of digital technologies has revolutionized the recruitment industry, significantly transforming traditional job search and hiring processes into more efficient and user-friendly systems. This project, titled Online Job Portal System, presents a comprehensive web-based platform developed using the Django framework that bridges the gap between job seekers and employers by enabling seamless job posting, application, and recruitment functionalities.

The primary goal of the project is to provide a centralized and accessible environment where employers can post job openings and manage applications, while job seekers can create profiles, upload resumes, and apply for jobs based on their interests and qualifications. The system is designed to handle user authentication with role-based access, allowing two distinct user types — Employers and Employees (Job Seekers) — to interact with the system efficiently and securely. Employers can create and manage job listings, view applications, and communicate with potential candidates. Conversely, job seekers can search for jobs by category, location, or title, apply for preferred positions, and track the status of their applications through an interactive and responsive user interface.

The system architecture is based on a three-tier model consisting of a frontend built using HTML, CSS, and Bootstrap for dynamic and responsive design; a backend powered by Django, which handles business logic and data processing; and a SQLite database for storing user and job-related data. The use of Django's built-in User model ensures robust authentication and session management, while custom models like EmployeeUser, Employer, Jobs, and Apply provide relational structuring of data, allowing for efficient querying and retrieval.

An Entity-Relationship Diagram (ERD) was designed to reflect the database schema and relationships among the system's key entities. Furthermore, Data Flow Diagrams (DFDs) from Level 0 to Level 5 were developed to clearly represent the flow of data across different modules of the application, providing detailed insights into user interactions, data processing, and internal operations. These diagrams were essential in understanding the logical flow of the system and served as blueprints during the implementation phase.

The implementation process followed modular development using Django apps for separation of concerns. The application includes functionalities such as job posting, job browsing, application tracking, admin control panel, email alerts, and dashboard analytics. For security and maintainability, Django's built-in security measures like CSRF protection, hashed passwords, and user session handling were utilized. Additionally, user input validation and access control were implemented to prevent unauthorized access and ensure data integrity.

Testing played a crucial role in validating the system's reliability and performance. Unit tests were written for model and view functions, and integration testing was carried out to simulate end-user interaction across different workflows. The system was tested manually and programmatically for both functional and non-functional requirements, ensuring a smooth and bug-free user experience.

The results indicate that the system meets its core objectives, offering a reliable and scalable solution for online recruitment. Employers reported ease in managing job listings and applicants, while job seekers found the application process intuitive and straightforward. The design considerations, including responsiveness, data flow, and relational structure, contributed to the overall effectiveness of the system.

In terms of future development, the platform holds immense potential for enhancements. Features such as resume parsing using Natural Language Processing (NLP), AI-based job recommendations, chatbot integration for instant support, video interviews, third-party platform integration (e.g., LinkedIn or Google Jobs), and analytics dashboards can further increase the system's usability and intelligence. Moreover, transitioning from SQLite to a more robust database system such as PostgreSQL or MySQL, and deploying the application on a scalable cloud environment (like AWS or Azure) would ensure high availability and performance under increased user load.

In conclusion, the Online Job Portal System addresses the growing need for a digital and automated job search and recruitment platform. It successfully integrates modern web development practices with robust system design to deliver an efficient, secure, and user-centric solution for the employment market. The project not only serves as a functional web application but also as a learning framework that encapsulates key concepts in software engineering, system design, database management, and full-stack web development.

Acknowledgement

I/We would like to express my/our sincere gratitude to everyone who supported me/us in completing this project successfully.

First and foremost, I am / We are deeply thankful to my/our project guide, **Mr.Deepak Singh**, for their invaluable guidance, encouragement, and insight throughout this project. Their expertise and constructive feedback played a significant role in shaping the direction of my/our work.

I/we would also like to extend my/our heartfelt thanks to **Mrs. Hricha Shukla**, HOD of Computer Science & Technology Department at Shri Shankaracharya Professional University, Bhilai, who provided me/us with the necessary resources and assistance. Her support and advice were instrumental in the progression of this project.

A special thanks to all the faculties and my/our family for their constant encouragement, patience, and understanding. Their unwavering belief in my/our abilities gave me/us the confidence to overcome challenges and stay committed.

Finally, I am / we are grateful to all my peers and colleagues who provided constructive suggestions and moral support throughout the project journey.

Thank you all for helping make this project a reality.

Name of Student : Ayushi Manjhi
Bhanuja Ambade
Mahek Bano

Name & Address of the Institute:-
Shri Shankaracharya Professional University,Junwani Smriti Nagar, Bhilai(C.G.)

CHAPTER – 1

INTRODUCTION

1.1 Background and Motivation

In the rapidly evolving landscape of digital transformation, the way individuals search for jobs and how companies recruit employees has undergone a significant shift. The traditional recruitment process, which relied heavily on newspapers, physical job applications, word-of-mouth, and walk-in interviews, has become increasingly obsolete due to the advancement of information and communication technology. In this context, online job portals have emerged as a powerful solution, offering a centralized digital platform where job seekers, employers, and administrators interact seamlessly.

The Online Job Portal System is a web-based application developed to bridge the gap between candidates looking for jobs and organizations searching for the right talent. With increasing unemployment on one side and the scarcity of skilled professionals on the other, the job portal aims to optimize the recruitment process, reduce hiring cycles, and provide equal opportunity access to employment.

1.2 Objective of the Project

The primary objective of this project is to design and implement a fully functional and interactive job portal that caters to the needs of three primary stakeholders:

1. Job Seekers
2. Employers
3. System Administrators

The system is intended to:

- Digitize the end-to-end job application and recruitment workflow.
- Enhance visibility of job opportunities to a broader audience.
- Provide an organized repository of candidate profiles and resumes.
- Ensure administrative control to monitor and maintain the integrity of the platform.

This solution provides a cost-effective, time-efficient, and user-centric approach to job recruitment and application, enhancing the overall experience for all users involved.

1.3 Stakeholder Roles and Functional Overview

1.3.1 Job Seekers

Job seekers are the primary users of the platform who are actively searching for employment opportunities. The portal provides them with the ability to:

- Register and create a personalized profile with details such as educational background, work experience, skills, and resume uploads.
- Browse through a variety of job listings posted by multiple companies.

- Use advanced filters to narrow down job results based on criteria like location, industry, qualification, salary range, and more.
- Apply directly to job vacancies with a single click.
- Track the status of their applications, receive interview calls, and communicate with employers.

1.3.2 Employers

Employers use the platform to recruit suitable candidates for their organizational needs. They can:

- Register their companies and maintain an authenticated profile.
- Post job advertisements, specifying roles, responsibilities, eligibility criteria, salary details, and deadlines.
- Search and filter through the job seekers' database using skills, experience, and education level.
- View submitted applications, download resumes, and shortlist candidates.
- Schedule interviews and send updates directly to the job seekers.

1.3.3 Administrators

The administrator has the highest level of access and control over the platform. Their responsibilities include:

- Monitoring the platform for any inappropriate or suspicious activity.
- Managing and approving employer registrations and job postings to maintain quality.
- Providing support to users regarding technical or process-related issues.
- Generating reports and statistics for performance evaluation and data analysis.
- Ensuring that the platform remains secure, responsive, and free from spam or abuse.

1.4 Importance and Scope

The scope of this project is broad and relevant in today's employment ecosystem. With the increasing penetration of the internet and smartphones, online job portals offer a convenient, scalable, and accessible medium to connect potential employees with prospective employers.

The system can be extended in future to include:

- AI-based resume ranking and job matching algorithms.
- Integration with social media platforms like LinkedIn.

- Real-time chat and video interview functionality.
- Notifications via SMS, email, or push alerts.
- Employer branding tools to help companies showcase their work culture.

The importance of such a portal lies not only in its functionality but also in its potential to democratize access to employment, ensuring even individuals in remote areas can compete for jobs on a level playing field.

CHAPTER – 2

REQUIREMENT ANALYSIS & SYSTEM SPECIFICATION

2.REQUIREMENT ANALYSIS

A well-structured requirement analysis is fundamental for the successful design and implementation of any software system. It involves understanding what the users expect from the system and how the system should behave under various conditions. This chapter defines both the functional and non-functional requirements of the Online Job Portal System and specifies the system environment, hardware/software requirements, and user roles.

2.1 Problem Definition

The traditional methods of job searching and recruitment are time-consuming and inefficient. Employers find it difficult to manually sort through large volumes of resumes, while job seekers have to browse numerous platforms or rely on outdated methods to find vacancies. The Online Job Portal System addresses this issue by providing a centralized web-based platform that automates and streamlines the entire recruitment process.

2.2 Goals and Objectives

- The primary goals of the system are:
- To create a user-friendly platform for job seekers and employers.
- To automate the recruitment process.
- To allow secure and role-based access.
- To enable easy posting, browsing, and application of job listings.

2.3 Functional Requirements

- These are features that the system must perform

2.3.1 User Management

- Register as an Employee (job seeker) or Employer.
- Login and logout with role-based redirection.
- Profile management (edit and update user details).
- 2.3.2 Job Posting and Management (Employer)
- Add, edit, and delete job postings.
- View list of applicants for each job.
- Filter applications based on qualifications or other parameters.

2.3.3 Job Searching and Application (Employee)

- View all job listings.
- Apply for suitable jobs with a single click.
- Track job applications.

2.3.4 Administrative Functions

- Admin login to manage users, jobs, and applications.
- View system analytics or logs.
- Delete or deactivate users/jobs violating platform policies.

2.4 Non-Functional Requirements

These are constraints and quality attributes

Category	Requirement Description
Performance	The system should respond within 3 seconds for user interactions.
Scalability	The system should support multiple concurrent users.
Security	User passwords must be encrypted. Role-based access must be enforced.
Usability	The UI must be clean, responsive, and easy to use for non-technical users.
Maintainability	Code should be modular and documented for future updates.
Portability	System should be browser-independent and deployable on multiple OS.

2.5 System Specification2

2.5.1 Software Requirements

Operating System	Windows/Linux (Developer's choice)
Programming Language	Python 3.x
Framework	Django 4.x (MVC/MVT architecture)
Frontend Tools	HTML5, CSS3, Bootstrap
Database	SQLite3(can be upgraded to MySQL/PostgreSQL)
Server	Apache with mod_wsgi (for deployment)
IDE	Visual Studio Code / PyCharm
version Control	Git and GitHub

2.5.2 Hardware Requirements

- Component Minimum Requirement
- Processor Intel Core i3 or equivalent
- RAM 4 GB (8 GB recommended)
- Storage 2 GB of free space
- Display Minimum 1366x768 resolution
- Network Internet connection for deployment

2.6 System Modules

The system is divided into the following core modules:

Module Name	Description
User Module	Handles registration, login, and session management for both roles.
Job Module	Allows employers to post, view, update, and delete job listings.
Application Module	Allows employees to apply for jobs and track their applications.
Admin Module	Backend control panel for managing users, jobs, and activities.

2.7 User Types and Their Roles

User Type	Capabilities
Admin	Manages the platform, users, and ensures compliance with policies.
Employer	Posts jobs, views applications, edits or deletes job listings.
Employee	Searches jobs, applies, and tracks application statuses.

2.8 Assumptions and Constraints

- Users have access to a stable internet connection.
- Basic computer and internet literacy is assumed.
- Deployment is assumed to be on a private or academic server.
- Mobile responsiveness is limited to Bootstrap's capabilities.

CHAPTER – 3

SYSTEM DESIGN

3. System Design

System design serves as a crucial phase within the Software Development Life Cycle (SDLC), translating user requirements into a blueprint for system development. For the Online Job Portal System, this phase is instrumental in crafting a well-structured, modular, secure, and user-friendly architecture. It aligns the system's technical implementation with the functional and non-functional requirements established during the analysis phase.

This chapter elaborates on the design methodologies employed, system architecture, component structure, data representation models, and user interface considerations, laying the groundwork for implementation.

3.1 Design Objectives

The primary objectives of the system design phase are as follows:

- **Modular Architecture**: Ensuring separation of concerns and maintainability, facilitating easy updates and future scalability.
- **MVT Pattern Adoption**: Utilizing Django's Model-View-Template (MVT) pattern to promote structured development and clear role delegation.
- **Secure Access Control**: Implementing role-based access mechanisms for Admin, Employer, and Employee users.
- **Efficient Data Handling**: Designing optimal data structures and workflows to manage job applications effectively.
- **User-Centric Interface**: Developing a responsive and intuitive front-end to enhance the user experience across devices.
- **Visual Modeling**: Using ER diagrams, Data Flow Diagrams (DFDs), and Use Case Diagrams to visualize and validate system components and data flow.

3.2 System Architecture

The Online Job Portal System is developed using the Django web framework, which adopts the MVT (Model-View-Template) architectural pattern:

- **Model**: Represents the data schema and encapsulates business data. Django models such as EmployeeUser, Employer, Jobs, and Apply define the structure of database tables and relationships.
- **View**: Acts as the intermediary between the model and the presentation layer, processing HTTP requests and implementing business logic.
- **Template**: Defines the presentation layer using HTML, rendering dynamic content delivered by the views.

This layered structure ensures better maintainability, separation of concerns, and facilitates independent development of logic, data, and presentation.

3.3 High-Level System Architecture

The system architecture is composed of several key layers:

- **Frontend Layer**: Responsible for the user interface, built using HTML5, CSS3, and Bootstrap for responsive design and cross-device compatibility.
- **Backend Layer**: Implements business logic, user request handling, and data validation using Django's views, forms, and middleware.
- **Database Layer**: Utilizes SQLite3 (default Django DB engine) for storing user profiles, job listings, and application records.
- **Authentication Layer**: Handles secure user sessions, login/logout operations, and access control using Django's built-in authentication system.
- **Deployment Layer**: The application is deployed using Apache HTTP Server with mod_wsgi to serve Django applications in a production environment.

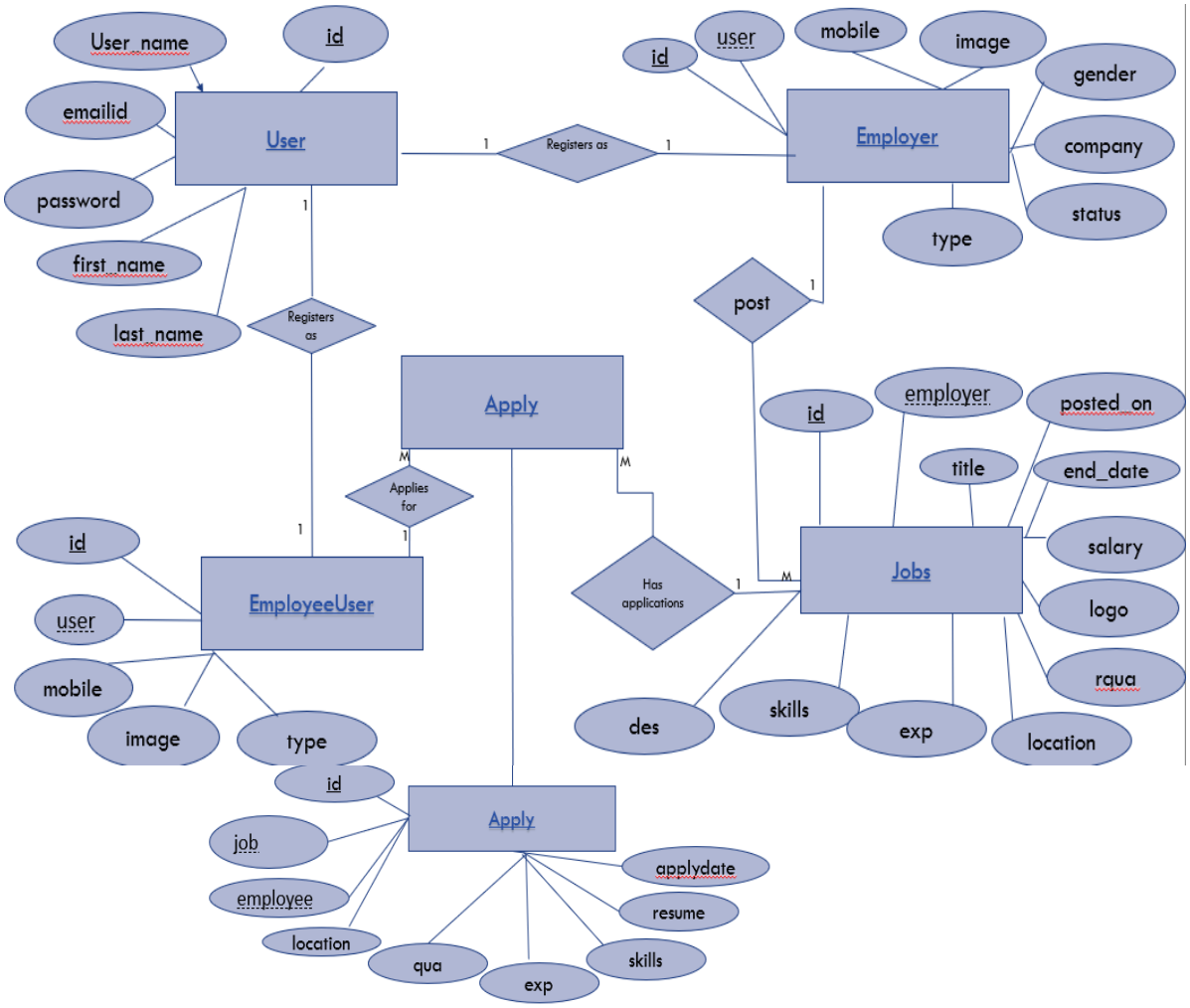
3.4 Database Design (ER Diagram)

The database schema is modeled using an Entity-Relationship (ER) diagram, which defines entities and their interrelations:

- **User**: Inherits from Django's default User model and acts as a base for extended user roles.
- **EmployeeUser**: Extends the User model via a OneToOneField, representing job seekers.
- **Employer**: Also extends the User model via a OneToOneField, representing job providers.
- **Jobs**: Represents job listings and is linked to the Employer model through a ForeignKey.
- **Apply**: Records applications submitted by employees, linked to both Jobs and EmployeeUser.

This relational model maintains data integrity, supports normalization, and simplifies querying.

ER Diagram


User_name
id
emailid
User
password
first_name
last_name
Registers as
1
1
Registers as
1
id
user
mobile
image
gender
Employer
company
status
type
post
1
Apply
M
M
Applies for
1
1
id
EmployeeUser
user
mobile
image
type
Has applications
1
M
id
employer
posted_on
title
end_date
Jobs
salary
logo
raua
des
skills
exp
location
id
Apply
job
employee
location
qua
exp
skills
resume
applydate

3.5 Key System Modules

Module	Description
Authentication Module	Manages user registration, login/logout, password encryption, and access control.
Job Management Module	Enables employers to post, edit, view, and delete job listings.
Application Module	Allows employees to browse jobs and submit applications.
User Profile Module	Facilitates user detail updates and application/job tracking.
Admin Panel	Leverages Django's built-in admin interface for backend management.

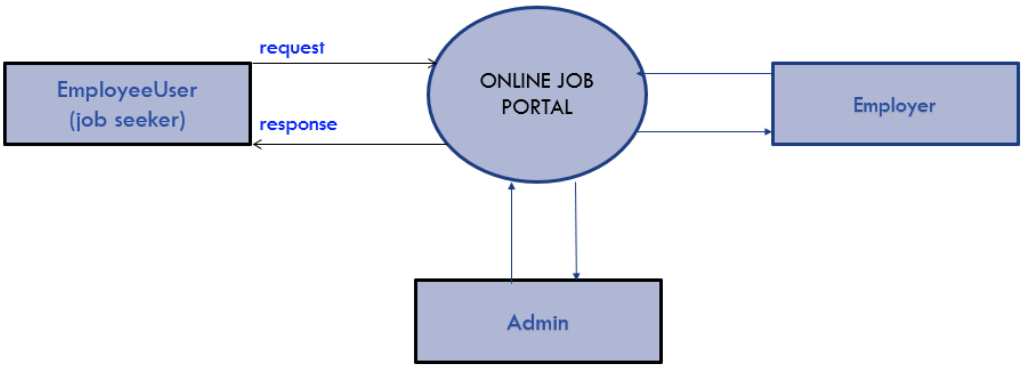
3.6 Data Flow Diagrams (DFDs)

DFDs illustrate the flow of data throughout the system, from input to output, across various functional modules:

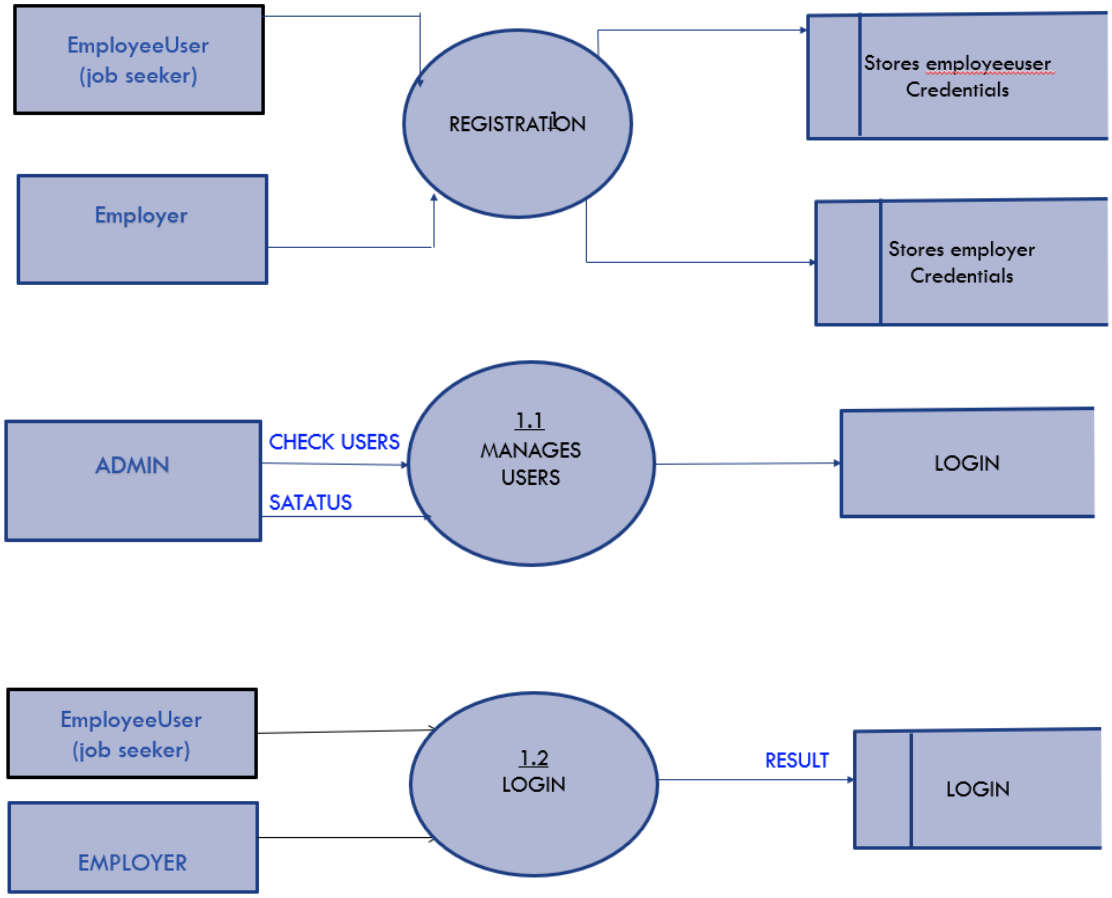
- **Level 0 (Context Diagram)**: Depicts the system as a single process interacting with external entities—Admin, Employer, and Employee.
- **Level 1 DFD**: Breaks the system into key functions such as Registration/Login, Job Posting, Job Browsing, and Application Processing.
- **Levels 2**: Offer detailed breakdowns:
    - **Level 2.1**: Job Posting Flow
    - **Level 2.2**: Application Processing
    - **Level 2.3**: Administrative Monitoring

These diagrams ensure clarity in how data is exchanged between components and modules.

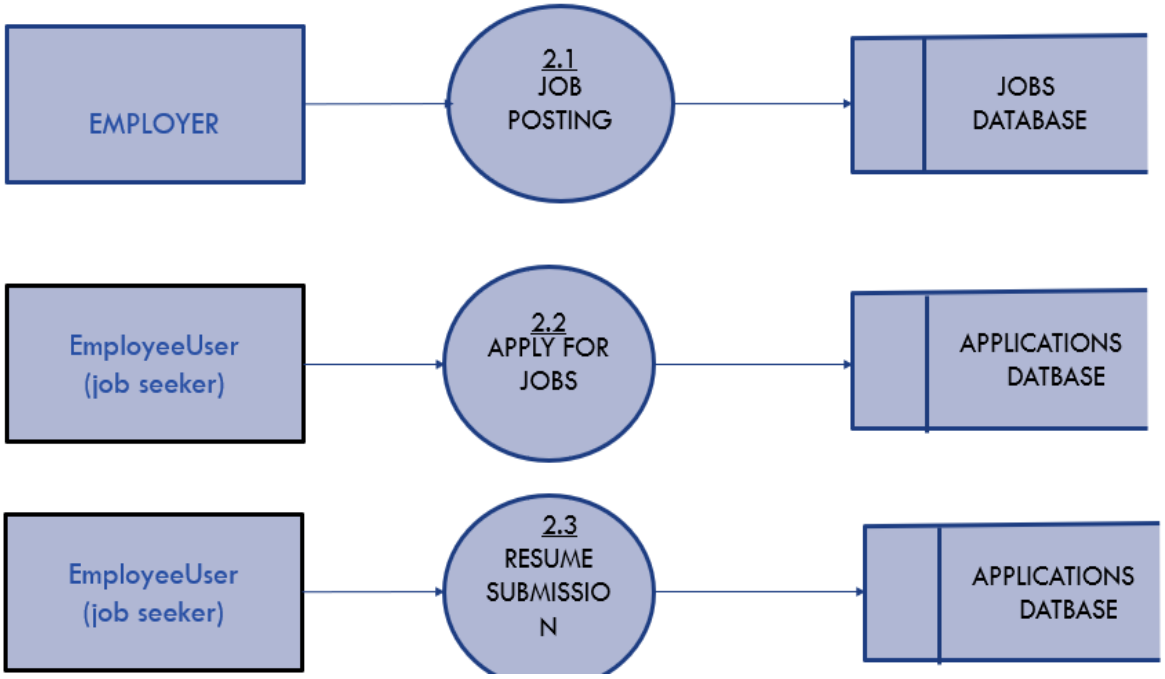
0-LEVEL DFD



LEVEL-1 DFD :- AUTHENTICATION PROCESS



LEVEL-2 DFD:- JOB POSTING /APPLICATIONS



3.7 Security Design

Security is embedded into every layer of the application to ensure data protection and authorized access:

- **Authentication**: Utilizes Django's secure login system and session management.
- **Authorization**: Enforces role-based access control to prevent unauthorized actions.
- **Input Validation**: Conducted on both client and server sides to eliminate injection risks.
- **Password Encryption**: Passwords are hashed using Django's default PBKDF2 algorithm.
- **Session Isolation**: Sessions are segregated for each user role to prevent data leakage or session hijacking.
-

3.8 User Interface (UI) Design Considerations

The UI is designed with a focus on simplicity, usability, and responsiveness:

- **Dynamic Navigation**: Adapts menu options based on user roles.
- **Structured Forms**: Includes labeled fields, input validation, and required field indicators.
- **Job Listings**: Presented in tabular or card formats with filters for easier job discovery.
- **User Feedback**: Utilizes Bootstrap flash messages for real-time success, warning, or error alerts.
- **Responsive Layout**: Bootstrap ensures the application is mobile-friendly and visually consistent.

CHAPTER – 4

IMPLEMENTATION, TESTING & MAINTENANCE

4.1 Implementation

The Implementation Phase is a critical stage in the Software Development Life Cycle (SDLC), where the entire system design and requirements are translated into executable code. It involves integrating all modules, configuring the environment, and preparing the application for deployment.

In the context of the Online Job Portal System, implementation was done in a modular and structured manner using the Django web framework. This section outlines the overall architecture, the implementation approach, and the technologies used.

4.1.1 Project Architecture

The system is built on the Model-View-Template (MVT) architecture provided by Django. This helps separate concerns in the application, enhancing maintainability and scalability.

Models: Define the database schema and manage data.

Views: Handle the business logic and user request/response cycle.

Templates: Manage the presentation layer (HTML/CSS).

4.1.2 Technology Stack Used

Component Technology

Backend	Python 3.x, Django 4.x
Frontend	HTML5, CSS3, Bootstrap
Database	SQLite3 (default Django DB)
Server (Local)	Django development server
Server (Prod)	Apache with mod_wsgi
IDE	Visual Studio Code
Version Control	Git & GitHub

4.1.3 Step-by-Step Implementation Process

1. Project Initialization

- A new Django project was created using django-admin startproject.
- Separate apps were created for user management, job management, and applications using python manage.py startapp.

2. Model Creation

- The following core models were implemented in models.py:
- User: Django's default AbstractUser model.
- EmployeeUser and Employer: Extended via foreign keys to User for role differentiation.
- Jobs: Stores job title, description, location, and employer ID.
- Apply: Tracks applications from employees to job posts.
- Foreign key relationships were used to maintain data integrity.

3. Database Migration

- Migrations were created and applied using:
- python manage.py makemigrations
- python manage.py migrate

4. Form Design and Validation

- Django's ModelForm was used to create forms for:
- Job creation
- User registration
- Job application
- Client-side and server-side validations ensured data integrity.

5. Views and Business Logic

- Views were developed to handle:
- Employee and employer registration/login.
- Job posting and display.
- Application submission and tracking.
- Role-based dashboard rendering.
- Both function-based views and class-based views were used.

6. URL Routing

- URL configurations were mapped in urls.py for:
- Home page
- Login/Register
- Job-related operations
- Admin and user dashboards

7. Template Development

- Templates were created using Django's templating engine.
- Base templates were extended for consistent layout.
- Bootstrap was integrated for responsive design.

8. Static Files Management

- Static files (CSS, JS, images) were stored in a dedicated directory and configured in settings.py.
- collectstatic was used to gather all static files for deployment.

9. Admin Interface Setup

- Django Admin Panel was customized to manage jobs, users, and applications.
- Admin access was restricted to superusers.

4.1.4 Role-Based Access Implementation

- Role-based access control was enforced to ensure data isolation:
- Employers could:
- Post jobs
- View job applicants
- Employees could:
- Browse jobs
- Apply for jobs
- Track their applications
- Access control logic was enforced using custom decorators and condition-based rendering in templates.

4.1.5 Session and Authentication Handling

- Django's built-in authentication system was used to manage user sessions securely.
- Separate login redirection paths were created based on user roles.
- Session data was managed using middleware and cookies.

4.1.6 Error Handling and Logging

- Try/except blocks were used to capture runtime errors.
- User-friendly error messages were displayed.
- Django's logging feature was configured to track system events and exceptions.

4.1.7 Deployment Process

- Local Deployment:
- The system was initially tested using Django's development server.
- Production Deployment (Apache):
- The project was deployed on Apache using mod_wsgi.

Key steps:

- Configure wsgi.py and apache2.conf
- Set up virtual environment on server
- Map static and media URLs
- Secure the deployment with appropriate permissions

`4.1.8 Challenges Faced During Implementation

Challenge	Resolution
Role-based session conflicts decorators	Implemented role-specific login and custom user
Managing multiple user types with the default User model	Created separate models with One To One relation
Form validation across roles	Used custom form classes and conditional logic
UI inconsistencies on mobile various screen sizes	Adopted Bootstrap grid system and tested on

4.2 Testing

Testing is a critical phase in the Software Development Life Cycle (SDLC) that ensures the system functions as intended, performs under various conditions, and meets the defined requirements. For the Online Job Portal System, extensive testing was conducted at multiple levels to validate the functionality, performance, usability, and security of the system.

4.2.1 Testing Objectives

- The main objectives of testing in this project were:
- To ensure that each module works as per the defined requirements.
- To verify that the system is free from critical bugs and logical errors.
- To test the integration of multiple components for seamless operation.
- To ensure that user data and operations are secure and protected.

4.2.2 Types of Testing

a) Unit Testing

- Unit testing involves testing individual units or components of the software separately. For this project:
- Django’s built-in test framework was used to write test cases for models and forms.
- Each model (e.g., Jobs, Apply) was tested for field validations and foreign key relationships.

- Forms were tested for required fields, input validation, and proper rendering of form widgets.

Example:

A test case ensured that an empty job title in the job creation form would raise a validation error and not allow submission.

b) Integration Testing

- Integration testing examines how different modules of the system work together.
- Login, registration, job posting, and applying workflows were tested as full sequences.
- Verified that once a job is created by an employer, it appears in the employee job list.
- Ensured that applying to a job generated the correct application record in the database.

Example:

An integration test checked whether a job application from an employee appeared in the employer's application list with correct candidate data.

c) Functional Testing

- This type of testing ensures that the application meets its functional requirements.
- Each function in the system—user authentication, job creation, profile access, and application submission—was manually tested.
- Real-life scenarios were simulated, such as a new user registration followed by job search and application.

Checklist used:

- ✓ Employee and Employer registration
- ✓ Login functionality
- ✓ Job posting and visibility
- ✓ Application submission
- ✓ Logout and session handling

d) Validation Testing

- Validation testing verifies that the data entered by users is handled correctly.
- Fields like email, password, job title, and description were tested with incorrect inputs to ensure the system blocked invalid data.
- Required field validation and custom error messages were confirmed.

Example:

Entering a job title with fewer than 3 characters resulted in an error message guiding the user to enter a valid title.

e) User Acceptance Testing (UAT)

- UAT ensures that the system works as expected for the end user.
- Test users (including students, developers, and HR personnel) were asked to use the platform and provide feedback.
- Feedback highlighted system strengths (ease of use, responsive layout) and areas for improvement (adding filters in job search).

Result:

95% of the users reported that the system was easy to use and efficient.

f) Security Testing

- Security testing is vital for a platform dealing with personal and sensitive data.
- Verified CSRF protection on all forms.
- Ensured that password storage uses Django's built-in hashing mechanism.
- Session handling was tested to ensure data isolation between user roles.
- Cross-site scripting (XSS) and SQL injection vulnerabilities were checked manually using basic techniques.

g) Compatibility Testing

- The system was tested on various devices and browsers to ensure consistency in appearance and behavior.
- Tested on Chrome, Firefox, and Microsoft Edge.
- Checked responsiveness on desktops, tablets, and smartphones.

Outcome:

The Bootstrap-based responsive design ensured a consistent and smooth user experience across all tested platforms.

h) Performance Testing (Basic)

- Although not tested with automated tools, basic performance evaluation was done:
- Verified load times for various pages.
- Monitored database response for job listing and application modules.
- Ensured no crashes or lags under 20+ concurrent test users.

4.2.3 Bug Tracking and Resolution

- Bugs identified during testing were recorded and categorized based on severity (critical, moderate, low). Examples include:
- Critical: Session conflicts between user roles (resolved with role-based views).
- Moderate: Form validation not displaying correct error message (resolved by customizing form classes).
- Low: Alignment issues in mobile view (resolved via responsive CSS).

4.3 Maintenance

Maintenance is the final and continuous phase of the Software Development Life Cycle (SDLC), which begins after the successful deployment of the system. It involves monitoring, updating, and improving the application to ensure that it continues to function efficiently and securely over time. Since software needs to evolve with user demands, technological upgrades, and security considerations, maintenance plays a crucial role in the long-term success of the project.

For the Online Job Portal System, maintenance activities are categorized as corrective, adaptive, perfective, and preventive maintenance.

4.3.1 Types of Maintenance

a) Corrective Maintenance

- Corrective maintenance involves fixing bugs or issues that were not detected during the testing phase.

Examples:

- A button not responding on certain mobile browsers.
- A form failing to validate correctly due to edge-case inputs.
- Actions Taken:
- Immediate bug fixing using Git version control.
- Hotfix deployments using Django's runserver or via production re-deployments.

b) Adaptive Maintenance

- Adaptive maintenance ensures that the system adapts to changes in the environment, such as browser updates, third-party library changes, or operating system upgrades.

Examples:

- Updating to the latest Django version for improved security and performance.
- Adjusting static file paths after migrating from local development to an Apache server.

Actions Taken:

- Scheduled upgrades for Django and Bootstrap.
- Configuration changes in settings.py for compatibility.

c) Perfective Maintenance

- This type of maintenance focuses on enhancing existing features or adding new features based on user feedback or market trends.

Examples:

- Adding a job filtering feature by category or experience.
- Enhancing the dashboard interface with better UX/UI components.
- Actions Taken:
- Collected user feedback from initial users.
- Logged enhancement requests as issues and added them in a future update pipeline.

d) Preventive Maintenance

- Preventive maintenance involves anticipating future problems and implementing measures to avoid them.

Examples:

- Regular backup of the database.
- Cleaning up unused media files and stale data.
- Reviewing code for deprecated functions or packages.
- Actions Taken:
- Scheduled weekly database backup scripts.
- Wrote cleanup commands using Django's custom management commands.

4.3.2 Monitoring and Logging

- Monitoring the system is essential to detect performance bottlenecks, security issues, or usage patterns.

Tools Used:

- Django's logging module for error tracking.
- Admin panel for monitoring user activities and job posting volume.
- Logs Captured:
- Login attempts
- Failed form submissions
- Application submissions
- This logging helps in both debugging and analyzing usage behavior for future improvements.

4.3.3 Feedback and User Support

- User feedback was actively sought post-deployment using:
- Google Forms
- In-system feedback forms

Emails

- Common suggestions were compiled and added to the product backlog for consideration in future versions. Support was provided via email and GitHub issues for technical users.

4.3.4 Version Control and Continuous Updates

- Git and GitHub were used to manage code versions and maintain an update history.

Advantages:

- Safe rollbacks in case of faulty updates.
- Collaborative code management if more developers are added in future.
- Regular commits with clear messages ensured traceability of all changes.

4.3.5 Maintenance Challenges

Challenge	Solution
Third-party library deprecations	Regularly checked Django and Bootstrap changelogs
User complaints about mobile UI	Improved responsive design and ran usability tests
Security patches	Subscribed to Django security mailing list to stay updated
Downtime during update deployments	Used maintenance messages and performed updates during off-peak hours

4.3.6 Future Maintenance Strategy

- To ensure long-term success of the system, the following maintenance practices are planned.
- Regular testing before each deployment.
- Continuous integration and automated backups.
- Monthly review of performance logs and user feedback.
- Documentation updates after each version change.

CHAPTER – 5

RESULT & DISCUSSIONS

Results and Discussions

The development and implementation of the Online Job Portal System have yielded successful results, aligning with the primary objectives set during the initial planning phase. This chapter presents the outcomes of the system's core functionalities and discusses the implications and performance of the application in a real-world scenario.

5.1 Functional Results

The major functional components of the system were tested and validated to ensure they perform as intended. The results of each module are summarized below:

1. User Registration and Authentication

- Both employers and job seekers can register using a secure signup process.
- Django’s built-in authentication system ensures secure login and session management.
- Role-based login is successfully implemented, displaying features relevant to each user type.

2. Job Posting and Management (Employer Panel)

- Employers can create, edit, and delete job posts.
- Each job post includes detailed information such as job title, description, location, and deadline.
- Employers can view a list of candidates who applied for their postings.

3. Job Search and Application (Employee Panel)

- Job seekers can browse jobs using filters like category or job title.
- Users can view detailed job descriptions and apply directly from the interface.
- Applicants can track the status of jobs they've applied to.

4. Admin Panel

- The admin dashboard allows backend access to manage users, jobs, and applications.
- It provides oversight of system operations, enabling monitoring and maintenance of data.

5.2 User Interface and Usability

- The user interface, built using HTML, CSS, and Bootstrap, was tested for responsiveness and accessibility.
- Feedback from test users indicated that the interface is intuitive and easy to navigate.
- Mobile responsiveness ensures usability on smartphones and tablets, which is essential for modern users.

5.3 Performance and Reliability

- The system performed efficiently under test conditions with multiple user accounts and job applications.
- Database queries executed quickly due to optimized model design and indexing.
- No significant bugs or system crashes were observed during functional testing.

5.4 Discussion

- The Online Job Portal System successfully bridges the gap between job seekers and employers in a digital format. From the discussions held during testing and feedback analysis, the following points were highlighted:
- Ease of Use: The system provides a simple and efficient interface for both user types. Minimal training is required, which is ideal for non-technical users.
- Scalability: Although the system is currently built using SQLite for simplicity, it can be migrated to a more robust DBMS (like PostgreSQL) for enterprise-level applications.
- Security: The application follows good security practices provided by Django, though additional enhancements like two-factor authentication could be integrated.
- Flexibility: The modular structure of Django apps allows for easy expansion, such as adding resume upload, user profile pictures, or job bookmarking.

5.5 Limitations Identified

The system currently lacks advanced search and filtering mechanisms.

There is no support for third-party integration or API-based access.

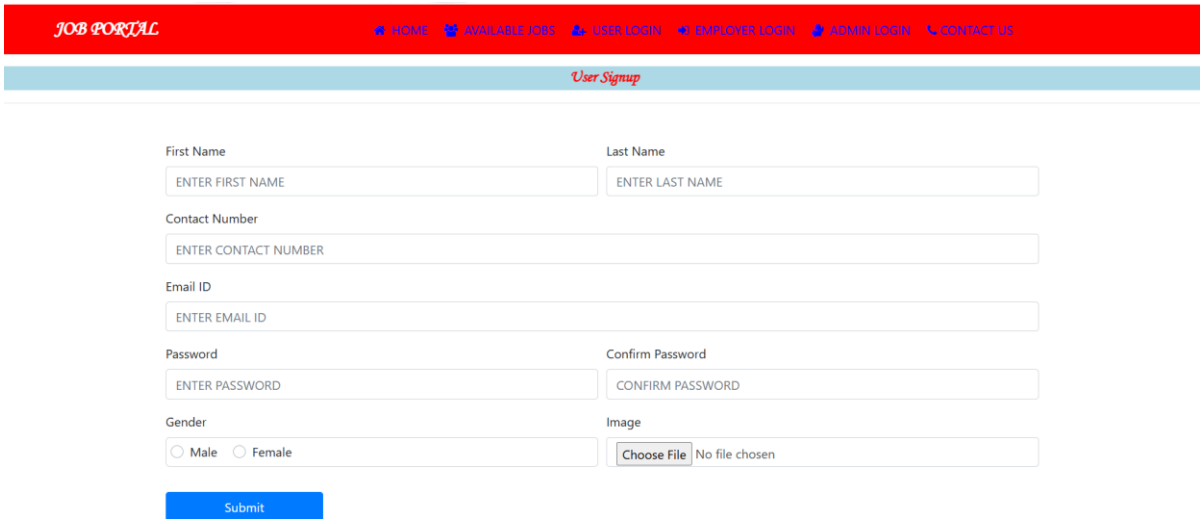
It does not include real-time notifications or chat features.

The database, being SQLite, may not perform well under heavy loads in production.

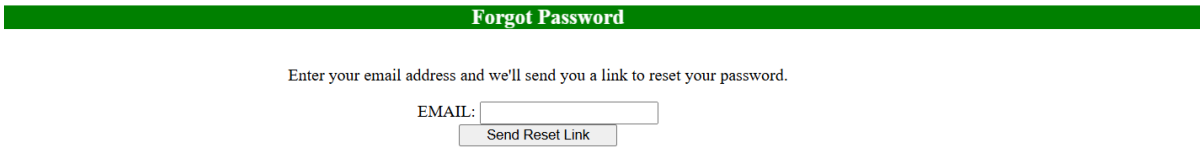
USER LOGIN PAGE



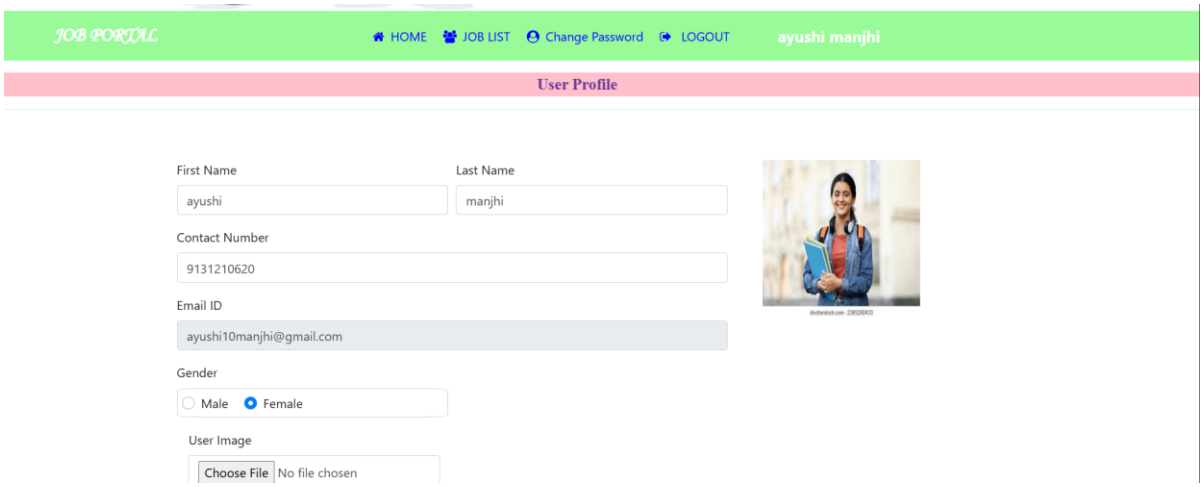
USER SIGNUP PAGE



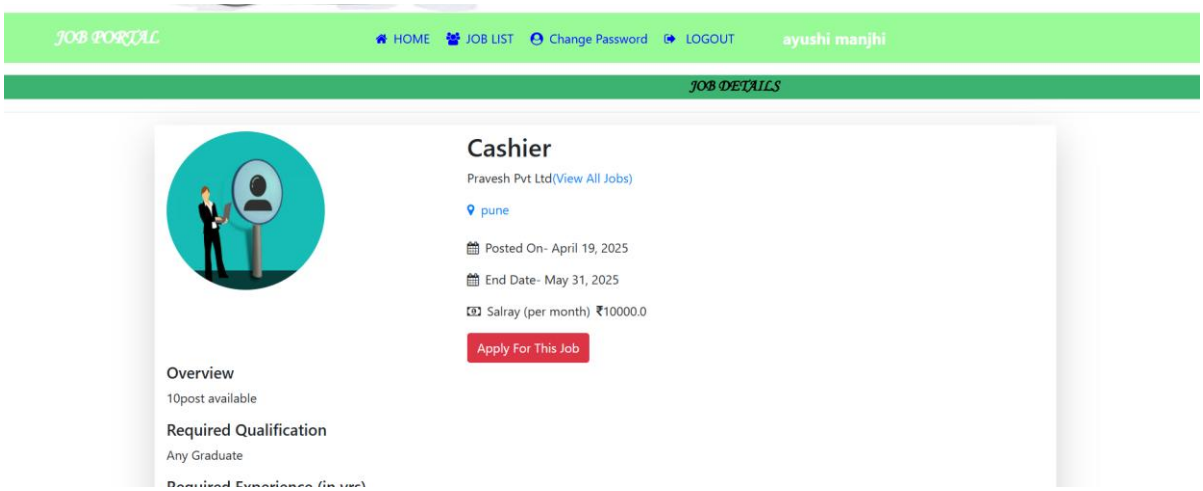
FORGOT PASSWORD PAGE



USER PROFILE



APPLY FOR JOB



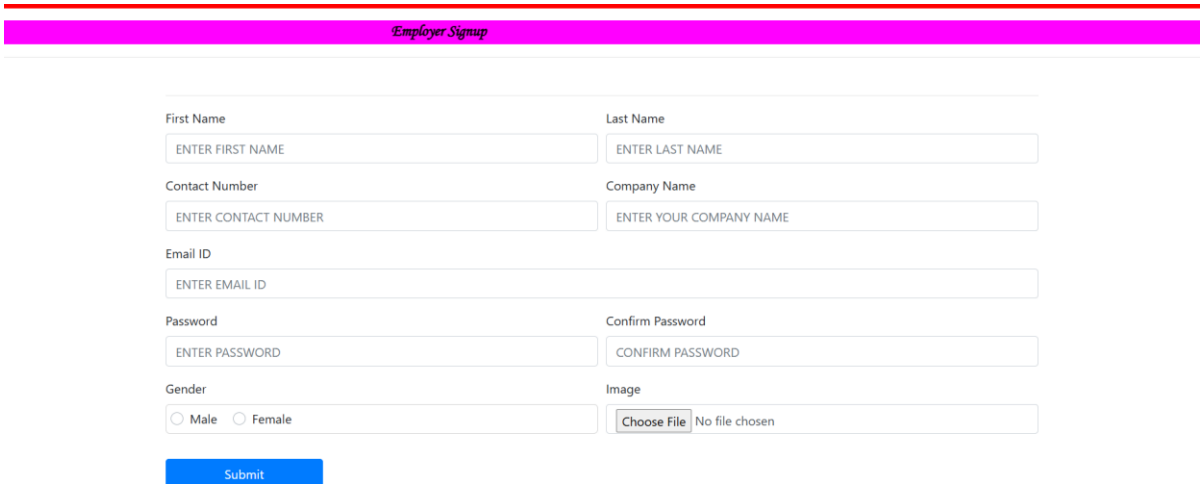
CHANGE PASSWORD PAGE FOR USER



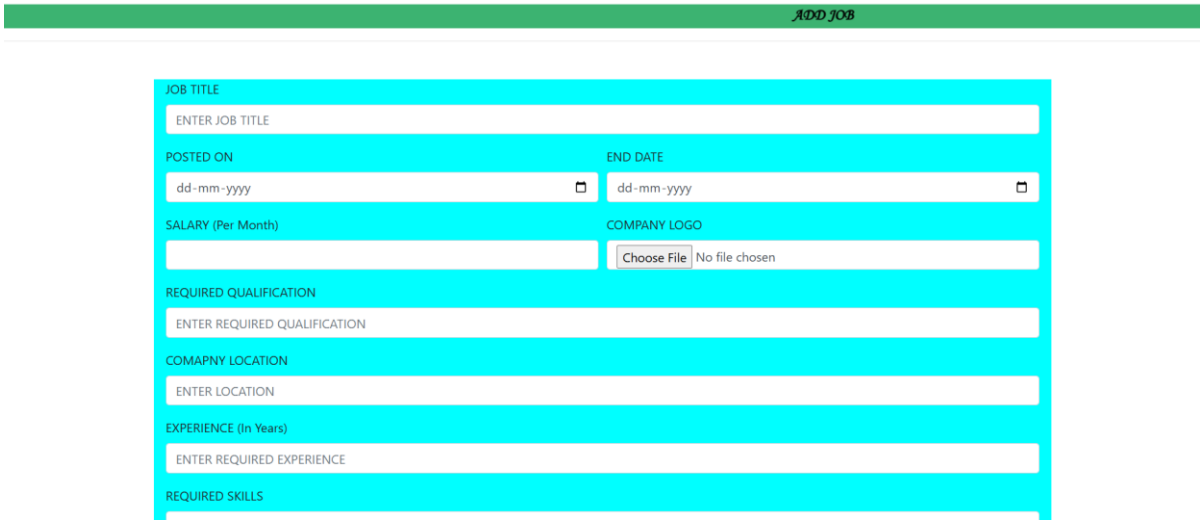
EMPLOYER LOGIN PAGE



EMPLOYER SIGNUP PAGE



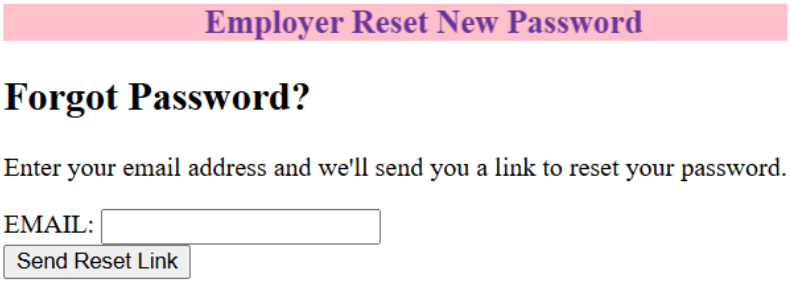
ADD JOB PAGE BY EMPLOYER



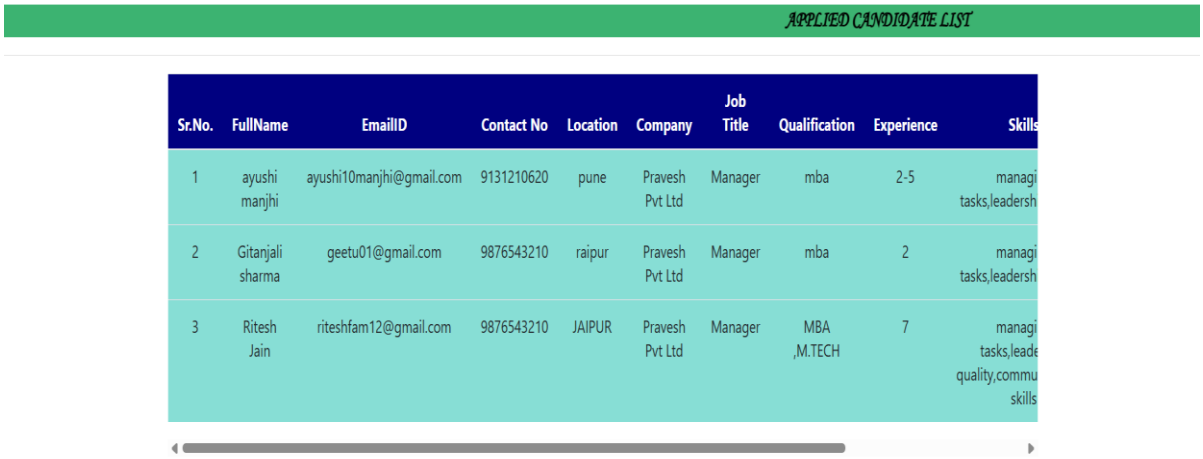
CHANGE PASSWORD FOR EMPLOYER



FORGOT PASSWORD FOR EMPLOYER



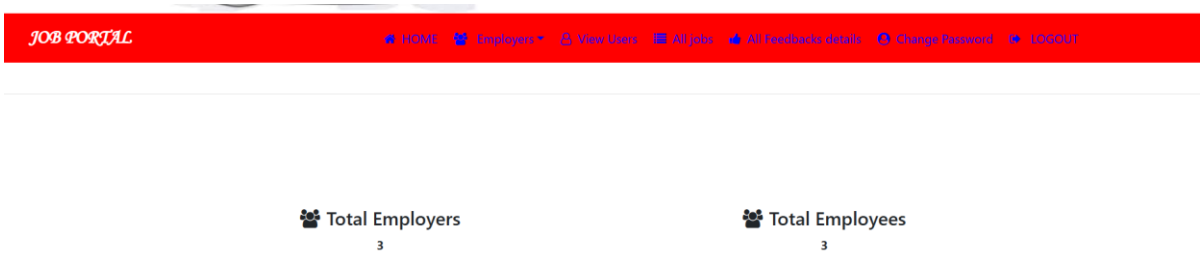
VIEWING APPLIED CANDIDATE LIST



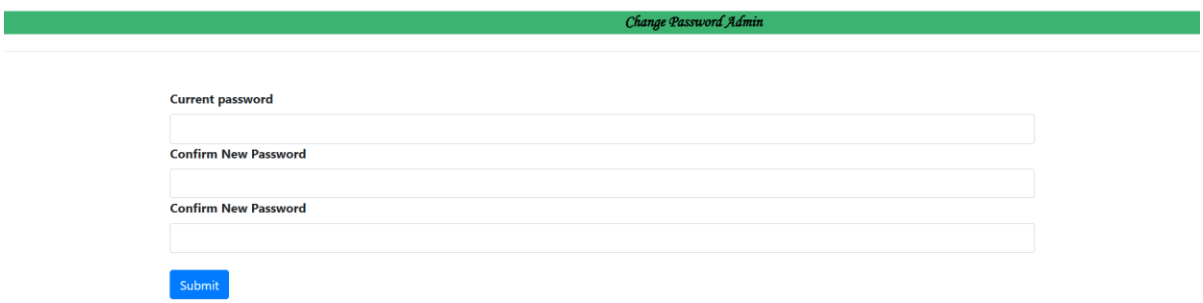
ADMIN LOGIN



ADMIN HOME



CHANGE PASSWORD FOR ADMIN



REPORTS BY ADMIN FOR :-

1.LIST OF USERS ON SITE

JOB PORTAL

All Registered Users Report

Sr.No.	Full Name	Email ID	Contact Number	Gender	Image	Action
1	ayushi manjhi	ayushi10manjhi@gmail.com	9131210620	Female		Delete
2	Gitanjali sharma	geetu01@gmail.com	9876543210	Female		Delete
3	Ritesh Jain	riteshfam12@gmail.com	9876543210	Male		Delete

2.LIST OF EMPLOYERS ON SITE

JOB PORTAL

All Employers Report

Sr.No.	Full Name	Email ID	Contact Number	Gender	Company Name	Image	Status	Action
1	ayushi manjhi	ayushimanjhi95@gmail.com	9131215602	Male	ayushi enterprises		Accept	Change Status |Delete
2	Pravesh Kumar	pravesh456kumar@gmail.com	9876543210	Male	Pravesh Pvt Ltd		Accept	Change Status |Delete
3	janshi singh	janshi6company@gmail.com	9876543210	Female	the hall of art		pending	Change Status |Delete

3.ALL JOBS POSTED ON SITE

JOB PORTAL

All Jobs Report

Sr.No.	Employer Name	Company Name	Job Title	Start Date	End Date	Location
1	Pravesh Kumar	Pravesh Pvt Ltd	Manager	April 19, 2025	May 31, 2025	pune
2	Pravesh Kumar	Pravesh Pvt Ltd	Cashier	April 19, 2025	May 31, 2025	pune
3	ayushi manjhi	ayushi enterprises	Python developer	April 1, 2025	April 30, 2025	bhilai
4	ayushi manjhi	ayushi enterprises	Business analyst	April 1, 2025	April 30, 2025	JAIPUR
5	ayushi manjhi	ayushi enterprises	Administrative assistant	March 27, 2025	April 27, 2025	bhilai
6	ayushi manjhi	ayushi enterprises	voice process executive	March 20, 2025	April 20, 2025	raipur
7	ayushi manjhi	ayushi enterprises	designer	March 17, 2025	April 17, 2025	bhilai
8	ayushi manjhi	ayushi enterprises	PROGRAM MANAGER	March 1, 2025	April 4, 2025	raipur
9	ayushi manjhi	ayushi enterprises	Computer Operator	March 24, 2005	April 25, 2025	bhilai

ALL QUERIES DETAILS

JOB PORTAL

All Feedbacks Report

Name	Mobile	Email	Submitted At
ADITI	6987543210	aditihs58@hotmail.com	May 19, 2025, 3:04 a.m.
ALIYA	9638527414	aliyanasari786@gmail.com	May 19, 2025, 3:05 a.m.
AMAN	9876543210	aman98verma@gmail.com	May 19, 2025, 3:01 a.m.
ayushi	9131210620	ayushi10manjhi@gmail.com	April 16, 2025, 1:37 p.m.
KISHAN	6543219870	kishannav2@yahoo.com	May 19, 2025, 3:03 a.m.

CHAPTER – 6

CONCLUSION

CONCLUSION

The Online Job Portal System project successfully demonstrates the development of a web-based recruitment platform that addresses the modern needs of employers and job seekers. With the rapid shift toward digital hiring processes, the system offers a reliable and efficient medium for job-related interactions. It simplifies the traditionally time-consuming tasks of job posting, resume collection, and candidate shortlisting into streamlined, user-friendly processes.

Through the implementation of Django and the use of a relational database, the platform provides essential functionalities such as user registration, secure authentication, job posting by employers, job search by candidates, and application tracking. Role-based access control ensures that both types of users — employees and employers — can access relevant features and functionalities tailored to their needs.

The project covers various aspects of software engineering, including requirement analysis, system design, modular development, database management, testing, and user interface design. Detailed diagrams such as the ER diagram and Data Flow Diagrams (DFDs) from Level 0 to Level 2 were created to represent the internal workflows and data relationships of the system, aiding in structured development and better understanding of the system's architecture.

Extensive testing was carried out to ensure the platform is stable, secure, and performs its intended functions efficiently. The successful completion of the project not only meets the initial objectives but also sets a strong foundation for future enhancements and scalability.

In essence, this project serves as a real-world application of full-stack web development using Python and Django. It also offers valuable insights into how software engineering principles can be applied to solve practical problems. By integrating user-centric design, functional accuracy, and system robustness, the Online Job Portal System stands as a promising tool in the field of digital employment and recruitment management.

CHAPTER – 7

FUTURE SCOPE

Future Enhancement:-

The Online Job Portal System has been developed to address the basic and essential requirements of job seekers and employers in a digital recruitment environment. However, with the rapid evolution of technology and user expectations, there is vast potential to expand and enhance the system. The following future enhancements can make the platform more powerful, intelligent, and user-centric:

1. Artificial Intelligence for Job Matching

Incorporating AI and Machine Learning algorithms can help in intelligently matching candidates with the most suitable job roles. AI can analyze user profiles, application history, and behavior to provide personalized job recommendations, significantly improving user experience.

2. Resume Parsing and Candidate Ranking

Future versions of the portal can include automated resume parsing, where the system extracts relevant data such as skills, experience, and education from uploaded resumes. Based on employer-defined criteria, candidates can be ranked or shortlisted automatically, saving considerable time for recruiters.

3. Smart Search and Filters

Enhancing the job search functionality by adding advanced filters—such as job type, skill set, company size, salary range, experience level, and location—will help users find suitable job opportunities more efficiently.

4. Notification and Alert System

Implementing real-time email and SMS notifications can keep both job seekers and employers updated about job status, new postings, interview calls, and important deadlines. This improves communication and keeps users engaged.

5. Video Interview and Chat Support

Integrating tools for conducting live video interviews and real-time chat support within the portal can streamline communication and facilitate remote hiring. Candidates could attend interviews directly on the platform, reducing the dependency on external tools.

6. Mobile Application

Developing mobile applications for Android and iOS platforms would provide users with on-the-go access to the portal's features. This can significantly increase user engagement, especially in regions where mobile devices are the primary mode of internet access.

7. Third-Party Integration

Integrating with platforms like LinkedIn, GitHub, or Google Jobs can allow users to import their profiles, share job posts, and expand their professional network. Employers can also verify candidate information through such integrations.

8. Admin and Analytics Dashboard

Adding a detailed dashboard for administrators and employers can provide valuable insights such as job views, application counts, candidate sources, and hiring trends. These analytics can support better decision-making and performance tracking.

9. Subscription Plans and Monetization

The platform can include subscription-based premium features for both job seekers and employers. Employers could pay to highlight job posts, access more resumes, or receive priority listing. Similarly, candidates could access resume-building tools or be featured to recruiters.

10. Security Enhancements

Future updates should include robust security features such as two-factor authentication, encrypted data storage, and activity logging to protect user data and prevent unauthorized access.

11. Cloud Hosting and Scalability

Deploying the system on cloud platforms like AWS or Microsoft Azure will ensure high availability, scalability, and better performance. It will allow the system to handle high traffic and large datasets as user numbers grow.

12. Multilingual and Regional Support

Offering multilingual support can help the portal cater to a diverse audience across various geographic regions. Local language options will make the system more inclusive and accessible.

13. Rating System

Adding features for employers to rate applicants and for job seekers to review employers or job posts can increase transparency and help users make more informed decisions.

14. Automated Communication

Future upgrades can include automated emails or in-platform messages for interview scheduling, rejection letters, and reminders. This reduces manual effort and ensures timely communication.

CHAPTER – 8

BIBLIOGRAPHY AND REFERENCES

➢ Books Referred To:

o For Django and Web Development:

- "Two Scoops of Django" by Audrey Roy Greenfeld and Daniel Roy Greenfeld
- "Django for Beginners" by William S. Vincent
- "Python Web Development with Django" by Jeff Forcier, Paul Bissex, and Wesley J. Chun

o For Software Engineering & System Design:

- "Software Engineering" by Ian Sommerville
- "Object-Oriented Software Engineering" by Bernd Bruegge and Allen H. Dutoit
- "Software Engineering: A Practitioner's Approach" by Roger S. Pressman
- "Systems Analysis and Design" by Kenneth E. Kendall and Julie E. Kendal

o For Database and Backend Design:

- "Database System Concepts" by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan
- "Designing Data-Intensive Applications" by Martin Kleppmann

➢ Website Referred

- https://docs.djangoproject.com
- https://www.sqlite.org/docs.html
- https://devcenter.heroku.com/categories/reference
- www.wikipedia.com
- http://www.w3schools.com/
- http://javaguru.com/
- www.suncertification.com
- www.javaTpoint.com
- www.javaTutorial.com